Data bases 2

Reliability Control

Reliability

For which transaction properties?

- Atomicity: all or nothing semantics
- Durability: persistence of effect

With which basic mechanisms?

- Stable memory → Backup
- Log for recovery

Backup mechanims

- Example in MySQL

https://dev.mysql.com/doc/refman/8.4/en/backup-types.html

Dimension	Main Options	Notes
Backup form	Physical / Logical	File copy vs. SQL export
Server state	Online (hot) / Offline (cold)	Live vs. stopped
Location	Local / Remote	On-host or over network
Technique	Snapshot / Full / Incremental	At specific time points/ Entire copy/ Only changes since last backup
Recovery	Full / Point-in-time	Log replay full or up to timestamp
Other functiontalities	Scheduling / Compression / Encryption	Usually external or enterprise tools

Buffer Management Policies

When to write modified ("dirty") pages back to disk?

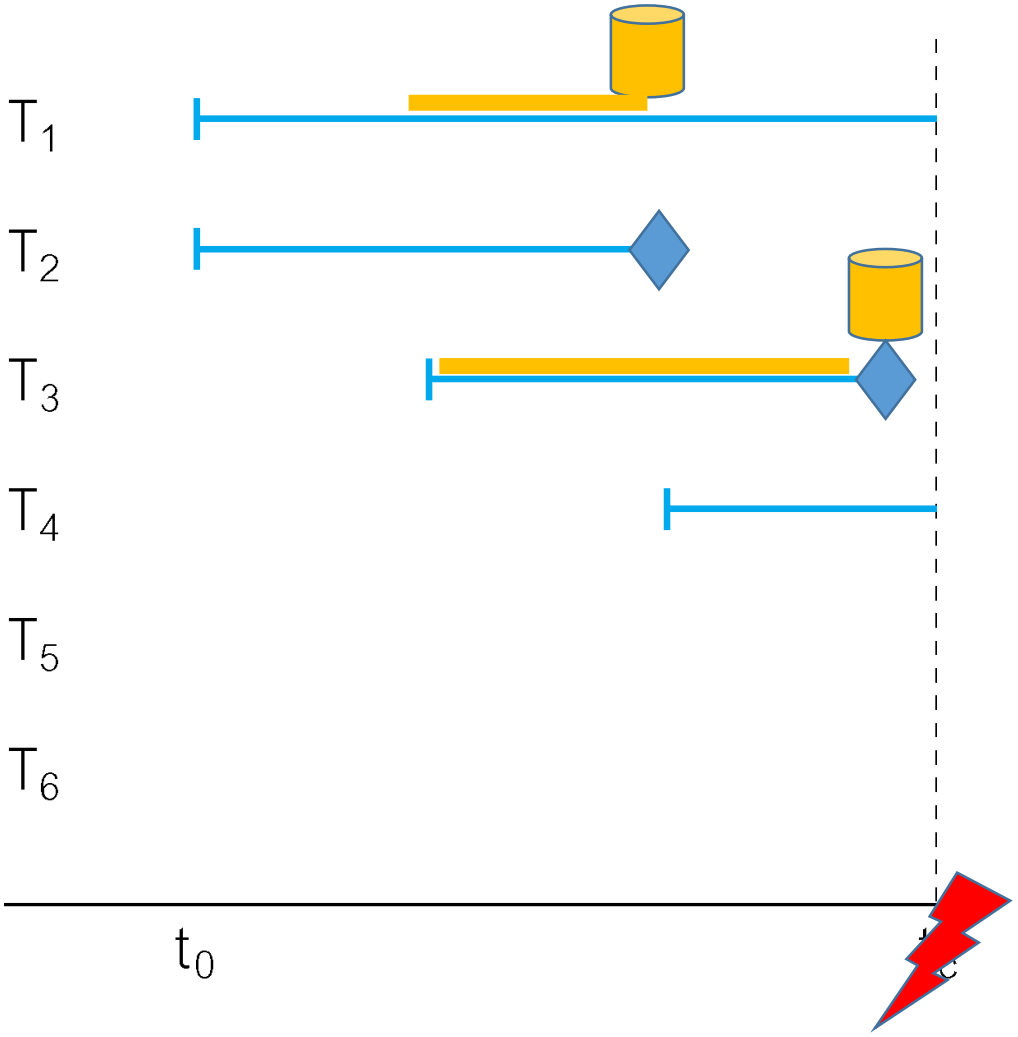
Policy	Description	Pros	Cons
FORCE	When a transaction commits, all its modified pages are *forced* to disk immediately.	Simpler recovery (no REDO needed).	Slower commits → must wait for disk I/O.
NO-FORCE	Committed changes may stay in memory; writing to disk is delayed until later (e.g., during checkpoint or buffer replacement).	Faster commits (asynchronous I/O).	Needs **REDO** during recovery, since committed data may still be only in RAM.

Buffer Management Policies

Which dirty pages can be "evicted" and written into the secondary memory?

Policy	Description	Pros	Cons
STEAL	The buffer manager may "steal" a dirty page even if the transaction that modified it hasn't committed yet	Reduces memory pressure (frees buffer space)	Requires **UNDO** during recovery, because uncommitted changes might reach disk
NO-STEAL	Dirty pages of uncommitted transactions cannot be evicted — they stay in memory until commit or rollback	Simplifies recovery (no UNDO needed)	May block new transactions if memory is full

Example



T1: Not committed but stolen pages!

T2: Committed but dirty pages have not been written!!

T3: Committed and written onto disk

T4: Not committed but everything only in main memory

LOG

```
BEGIN TRANSACTION
   UPDATE Accounts SET balance = balance – 100 WHERE account_id = 1;
   INSERT INTO Transactions(account_id, amount, type, date)
                        VALUES (1, 100, 'debit', CURRENT_DATE);
END TRANSACTION;
```

Log Sequence:

<BEGIN T1>

<T1, Accounts.balance(account_id=1), old=500, new=400>

<T1, Transactions, inserted(account_id=1, amount=100, type='debit', date=2025-11-13)>

<COMMIT T1>

Write-ahead log rule

Log Sequence

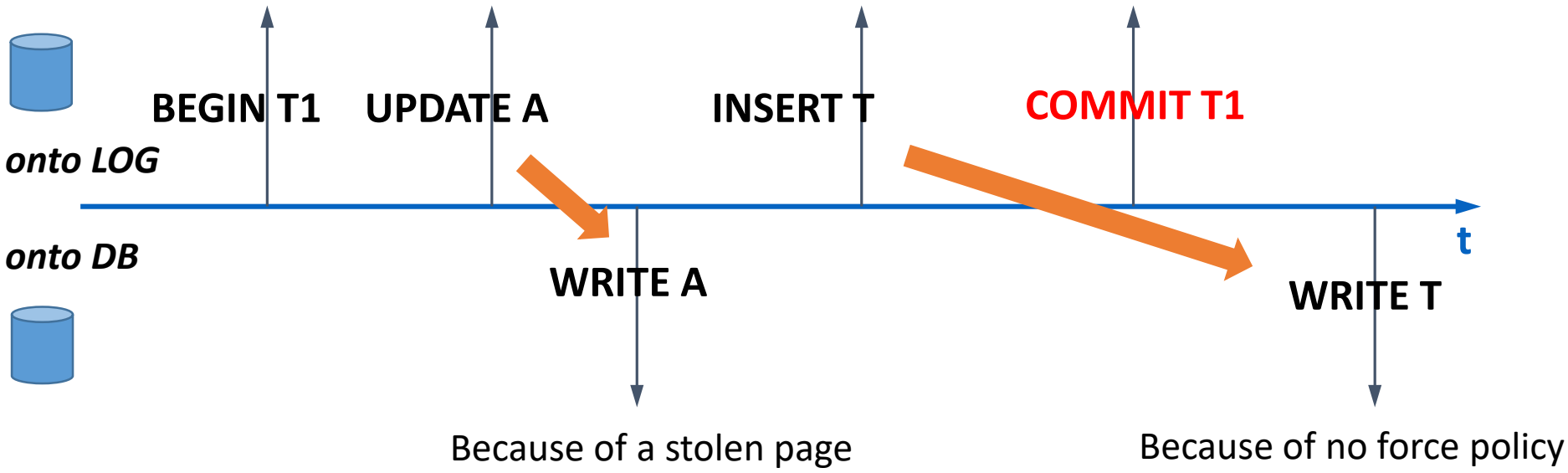
<BEGIN T1>

<T1, Accounts.balance(account_id=1), old=500, new=400>

<T1, Transactions, inserted (account_id=1, amount=100, type='debit', date=2025-11-13)>

<COMMIT T1>

WAL rule: Always write onthe log before changing the database itself



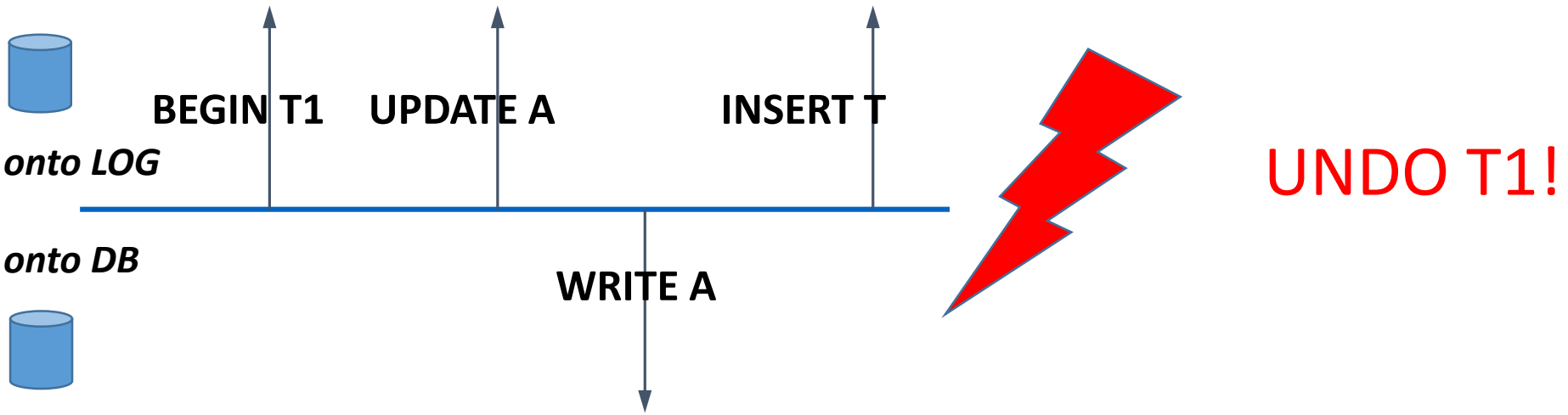
LOG and Write-ahead rule

Log Sequence

<BEGIN T1>

<T1, Accounts.balance(account_id=1), old=500, new=400>

<T1, Transactions, inserted (account_id=1, amount=100, type='debit', date=2025-11-13)>



Because of a stolen page

LOG and Write-ahead rule

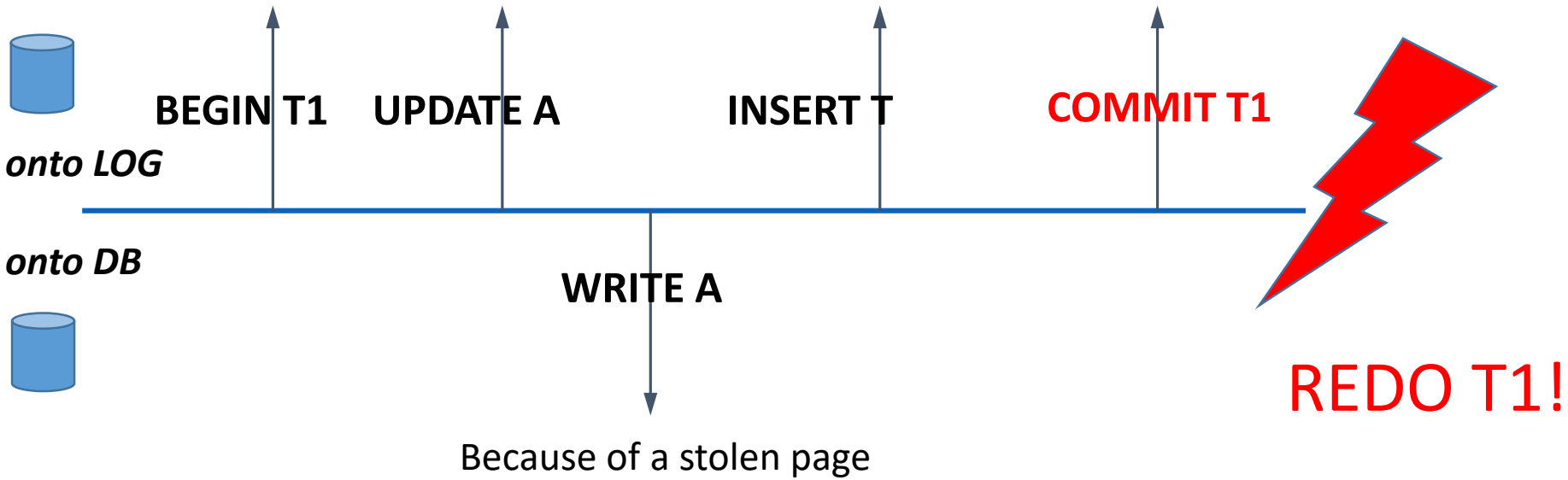
Log Sequence

<BEGIN T1>

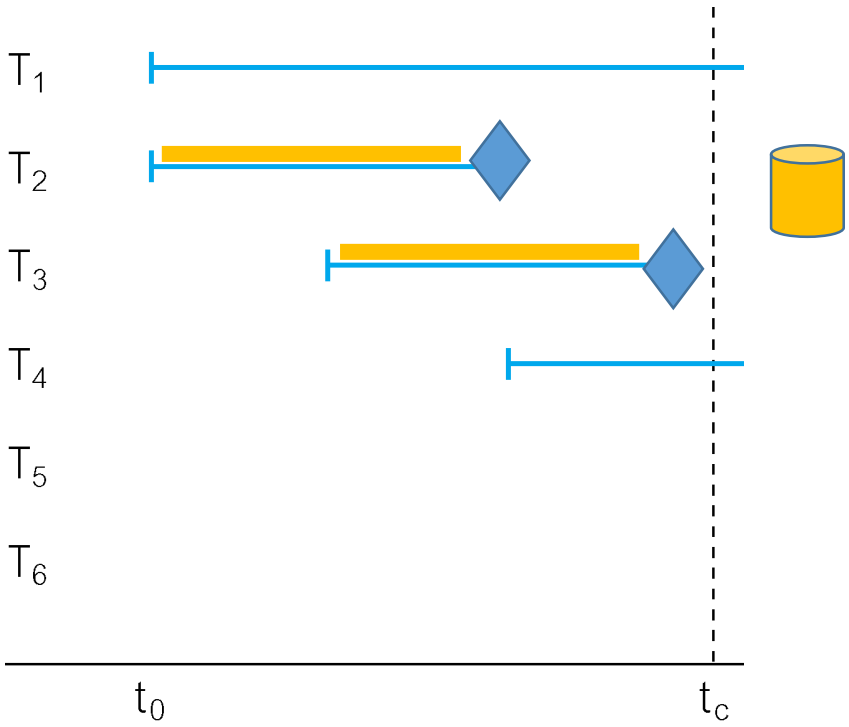
<T1, Accounts.balance(account_id=1), old=500, new=400>

<T1, Transactions, inserted (account_id=1, amount=100, type='debit', date=2025-11-13)>

<COMMIT T1>



Periodically: checkpoint

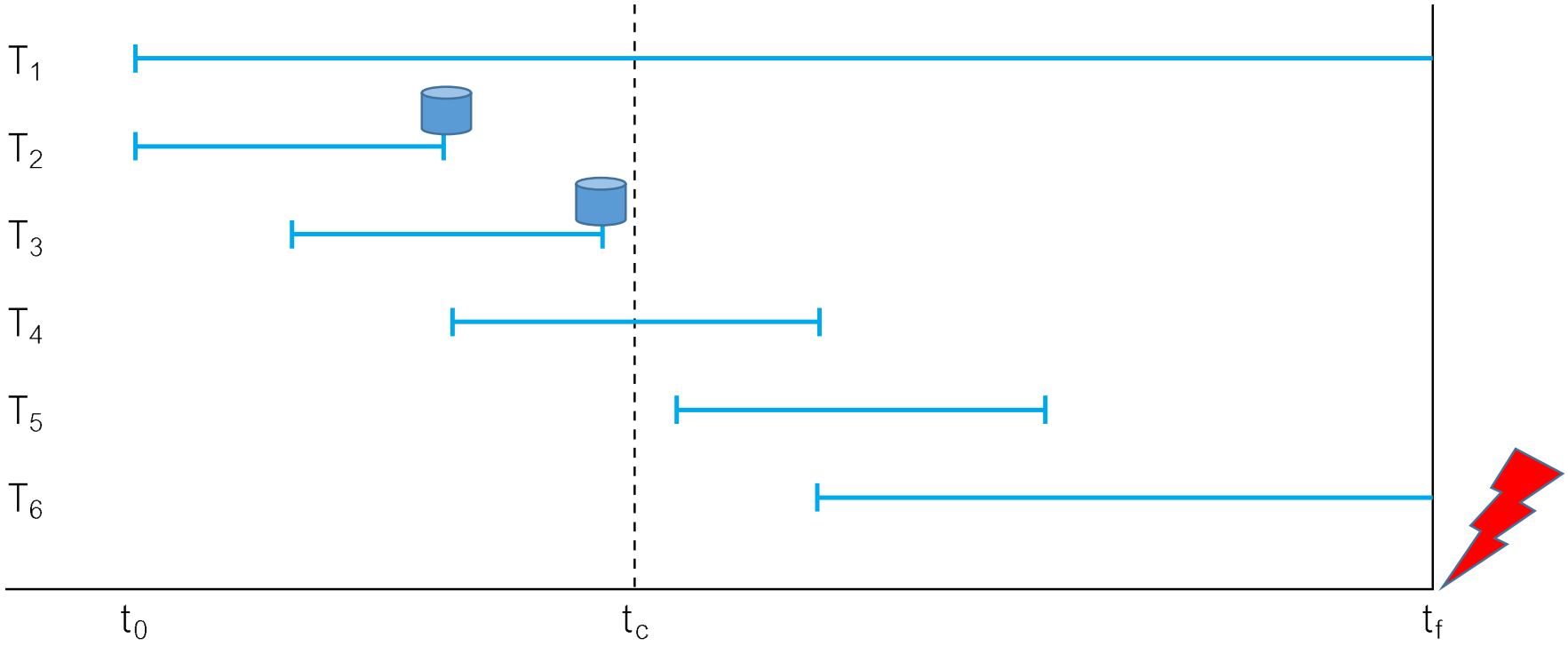


Active: T1, T4
Finished: T2, T3

- T2 and T3 already ended
  - They must be **written to secondary storage** (committed and permanently stored)
- T1 and T4 are in progress
  - List in the checkpoint active transactions
  - **CKPT (T_1, T_4)**

In **MySQL**: **fuzzy checkpointing** → The system gradually writes dirty pages to disk while transactions continue running.

Warm Restart



CKPT(T_1, T_4)

UNDO=(T_1, T_6)

REDO=(T_4, T_5)